SOMA SIMBÓLICA FORMAL EM HASKELL: ADIÇÃO DECIMAL VIA MÁQUINAS DE MEALY COM TABELA SIMBÓLICA PURA (SEM TIPOS NUMÉRICOS)

PETERSON CARARA JUNIOR

Prof. Responsável: João Paulo (JP)
FATEC Baixada Santista Rubens Lara – Ciência de Dados

SOMA SIMBÓLICA & A INTUIÇÃO NO PAPEL

A restrição do desafio

- **Proibição Absoluta de Tipos Numéricos:** é proibido utilizar `Int`, `Integer`, `Float` ou qualquer função de conversão (como `read` ou `show` de inteiros).
- **Pureza Estrita no Domínio dos Caracteres:** a máquina opera diretamente sobre o alfabeto `Char` (ex: `'0'..'9'`). Os números são fitas de texto puras.
- **A Pergunta Central:** como calcular `"999" + "1" = "1000"` se o processador não puder usar sua unidade lógica e aritmética (ALU)?

A analogia: soma no papel

- **Como uma Criança Soma no Papel?** Ela não calcula 78 + 56 de uma vez. Ela quebra em etapas elementares, coluna por coluna, da direita para a esquerda.
- **Memorização de Tabuada Fina (Memória Local):** ao ver '8' e '6', ela lembra: 'dá 4 e vai 1'. O 'vai-um' é a memória de curto prazo que viaja para a próxima coluna.
- **A Máquina de Estados:** o computador não precisa de números: só precisa de um estado que lembra se há 'vai-um' (`'1'`) ou não (`'0'`).

FUNDAMENTAÇÃO TEÓRICA: A MÁQUINA DE MEALY

Formalização: 6-tupla $M = (Q, \Sigma, \Lambda, \delta, \lambda, q_0)$

- $Q = \{'0', '1'\}$: conjunto finito de estados (Transporte: '0' = SemVaiUm, '1' = ComVaiUm)
- $\Sigma = Char \times Char$: alfabeto de entrada (o par de dígitos lido na coluna atual)
- $\Lambda = Char$: alfabeto de saída (o dígito impresso no resultado final)
- $q_0 = '0'$: estado inicial (iniciamos a soma sem transporte acumulado)
- $\delta : Q \times \Sigma \to Q$: função de transição de estado (calcula se haverá vai-um adiante)
- $\lambda : Q \times \Sigma \to \Lambda$: função de saída (emite o caractere resultante da coluna)

Por que Mealy e não Moore?

1. Saída Associada à Transição (Entrada + Estado): em Mealy, a saída depende do par lido E do estado atual simultaneamente. Isso permite emitir o caractere imediatamente.

2. Sincronismo Exato de Tamanho: para n pares de dígitos de entrada, uma máquina de Mealy emite exatamente n caracteres de saída durante o percurso.

3. Economia de Estados: uma máquina de Moore precisaria duplicar estados para cada saída possível (10 dígitos × 2 estados = 20 estados!). Mealy precisa de apenas 2.

Segurança Estática & Verificação

MATRIZ DE GARANTIAS FORMAIS EM HASKELL

Propriedade Teórica	Como o Haskell Viabiliza	Resultado Prático no Projeto
Completude Funcional	Pattern Matching Exaustivo	Zero omissões: o compilador recusa o código se faltar qualquer combinação de dígitos.
Segurança Estática (Type Safety)	Inferência e Verificação de Tipos (GHC)	Impossível misturar inteiros ou passar dados corrompidos para a função de transição.
Pureza Referencial	Funções Matemáticas Puras (Sem Efeitos)	Comportamento 100% determinístico: a mesma entrada gera sempre a mesma saída.
Simplicidade Algébrica	Equações Comutativas Diretas	Redução de 100 regras explícitas para 55 por simetria axiomática (a + b == b + a).
Totalidade Operacional	Ausência de 'error' ou 'undefined'	Para quaisquer duas Strings decimais válidas, o resultado é matematicamente garantido.

O GRAFO DE TRANSIÇÕES DA MÁQUINA DE MEALY

Estado 1: SemVaiUm (v = '0')

- Transição Interna (Loop): se $(c_1 + c_2) < 10$ → emite $c_1 + c_2$, permanece em v = '0'
- Transição para ComVaiUm: se $(c_1 + c_2) \geq 10$ → emite $(c_1 + c_2)$ mod 10, transita para v = '1'

Estado 2: ComVaiUm (v = '1')

- Retorno para SemVaiUm: se $(c_1 + c_2 + 1) < 10$ → emite $c_1 + c_2 + 1$, volta para v = '0'
- Transição Interna (Permanece em ComVaiUm): se $(c_1 + c_2 + 1) \geq 10$ → emite $(c_1 + c_2 + 1)$ mod 10, mantém v = '1'

Condição de parada e tratamento de overflow final

Quando as duas palavras de dígitos chegam ao fim (fita esgotada):

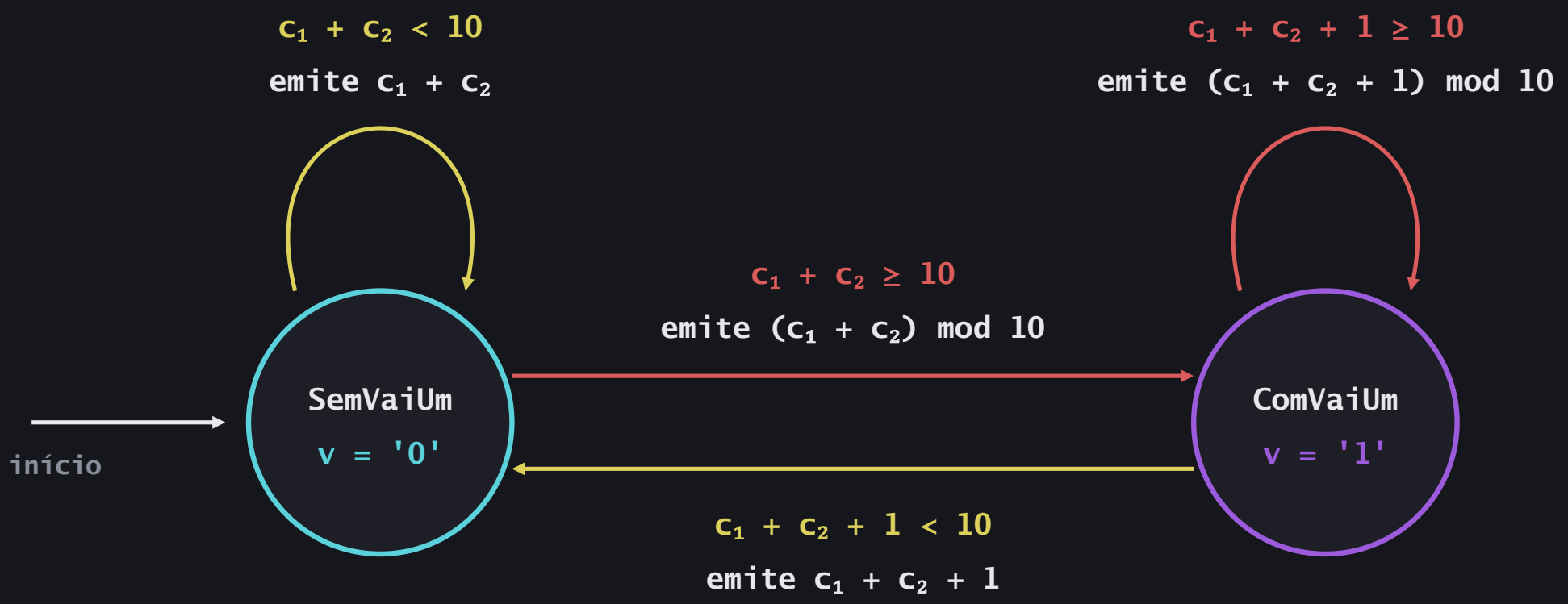
1. Se a máquina terminar em SemVaiUm (v = '0'): a computação encerra emitindo palavra vazia (ε). Não há dígito adicional.

2. Se a máquina terminar em ComVaiUm (v = '1'): o transporte residual vira o dígito mais significativo à esquerda ('1').

Exemplo clássico: "999" + "1" → colunas produzem "000", e o estado final '1' emite o prefixo '1' → "1000"!

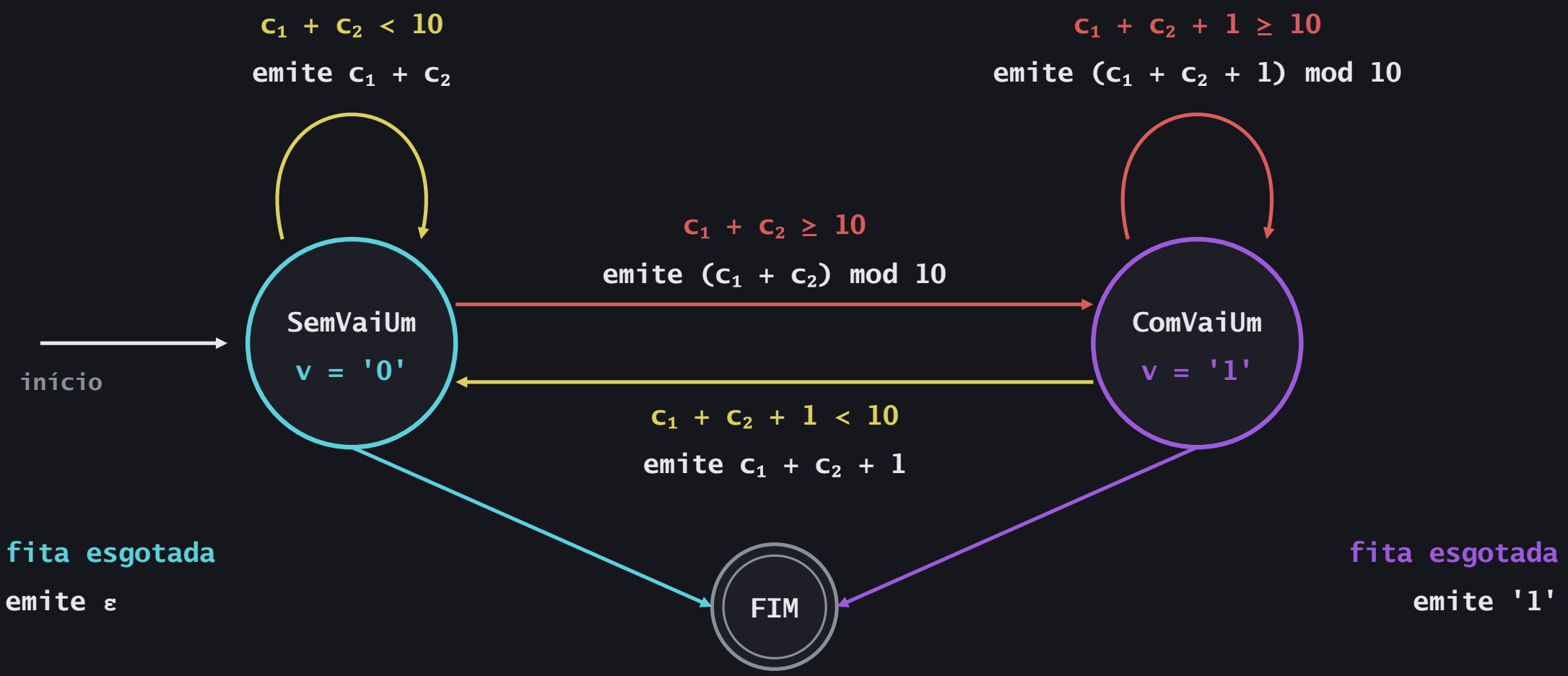
Arquitetura do Autômato

DIAGRAMA DE ESTADOS DA MÁQUINA DE MEALY



Aresta: condição sobre a coluna → dígito emitido. Σ = Char × Char; v é o transporte ('vai-um').

Fim da fita: em v = '1' emite '1' final à esquerda; em v = '0' emite ε.



Aresta: condição sobre a coluna → dígito emitido. Σ = Char × Char; v é o transporte ('vai-um').

FIM: marca só o término da fita (não é estado de aceitação): v = '0' emite ε; v = '1' emite o '1' final.

CÓDIGO PARTE 1: A TABELA SIMBÓLICA sd

SomaSimbolicaHaskell.txt — função sd

```haskell
-- sd :: Digito1 -> Digito2 -> VaiUm -> (Saida, NovoVaiUm)
sd :: Char -> Char -> Char -> (Char, Char)
sd '0' '0' '0' = ('0', '0')
sd '0' '0' '1' = ('1', '0')
sd '0' '1' '0' = ('1', '0')
sd '0' '1' '1' = ('2', '0')
-- ... (regras atômicas explícitas) ...
sd '8' '9' '0' = ('7', '1')
sd '8' '9' '1' = ('8', '1')
sd '9' '9' '0' = ('8', '1')
sd '9' '9' '1' = ('9', '1')

-- Redução axiomática por comutatividade:
sd a b v = sd b a v
```

Análise formal da tabela

- **Unificação de δ e λ em um Único Morfismo:** `sd` mapeia $(c_1, c_2, v) \mapsto (r, v')$. Saída e novo estado voltam em uma tupla atômica.
- **O Poder da Comutatividade (55 vs 100 regras):** a equação final `sd a b v = sd b a v` cobre os casos em que a > b, reduzindo a tabela pela metade!
- **Pureza Referencial Absoluta:** não há operação de soma matemática. O cálculo é uma consulta direta em uma tabela finita verificável por inspeção.

Propagação do Estado pela Fita

CÓDIGO PARTE 2: O MOTOR RECURSIVO sa

SomaSimbolicaHaskell.txt — função sa

```
sa :: String -> String -> Char -> String

-- Caso Base 1: Fitas vazias sem transporte
sa []     []     '0' = []

-- Caso Base 2: Fitas vazias com transporte pendente
sa []     []     '1' = ['1']

-- Tamanhos Desiguais (preenche com '0'):
sa (x:xs) []     v =
  let (r, nv) = sd x '0' v in r : sa xs [] nv
sa []     (y:ys) v =
  let (r, nv) = sd '0' y v in r : sa [] ys nv

-- Passo Indutivo Geral:
sa (x:xs) (y:ys) v =
  let (r, nv) = sd x y v in r : sa xs ys nv
```

Os 5 casos estruturais de sa

1. **Base Limpa ([], [], '0'):** as fitas acabaram simultaneamente e não há transporte. Retorna [].

2. **Base de Overflow ([], [], '1'):** as fitas acabaram, mas restou transporte. Emite o '1' final à esquerda.

3. e 4. **Alinhamento Desigual:** se um número tem mais dígitos que o outro (ex: 1000 + 5), o dígito ausente é tratado como '0' implícito.

5. **Passo Recursivo Geral:** desempacota (x, y), consulta sd, acopla r na cabeça e recursa na cauda.

Funções Públicas s e sl

CÓDIGO PARTE 3: ALINHAMENTO E GENERALIZAÇÃO

Alinhamento das unidades: função s

```
s :: String -> String -> String
s a b = reverse (sa (reverse a) (reverse b) '0')
```

- Por que inverter com reverse? Lemos números da esquerda para a direita, mas a soma manual ocorre da direita para a esquerda (pelas unidades).
- O Fluxo: 1. inverte as entradas ("123" vira "321"); 2. executa sa a partir do estado '0'; 3. inverte o resultado para restaurar a ordem.

Soma de múltiplos números: função sl

```
sl :: [String] -> String
sl [] = "0"
sl xs = foldl1 s xs
```

- O Poder da Alta Ordem com foldl1: acumula a operação binária s sobre uma lista de números, sem laços como while ou for.
- Exemplo de Redução Funcional: sl ["10", "20", "30"] = s (s "10" "20") "30" = s "30" "30" = "60". Generalização imediata para somar N números!

TRAÇO DE EXECUÇÃO DETALHADO: "87" + "45"

Passo	Operação Haskell	Entradas & Estado	Regra Consultada	Emissão & Novo Estado
1	reverse "87" / "45"	"78" e "54", v = '0'	Alinhamento inicial	Pronto para processar unidades
2	sd '7' '5' '0'	x='7', y='5', v='0'	sd '5' '7' '0' (comut.)	Emite '2', novo transporte v = '1'
3	sd '8' '4' '1'	x='8', y='4', v='1'	sd '4' '8' '1' (comut.)	Emite '3', novo transporte v = '1'
4	sa [] [] '1'	Fitas esgotadas, v='1'	Caso Base com Carry	Emite caractere extra '1'
5	Acumulação de sa	Resultado das chamadas	sa gerou: '2':'3':"1"	Palavra invertida: "231"
6	reverse "231"	Restauração da ordem	reverse "231"	Resultado Final: "132"

Análise Crítica & Epistemologia

O QUE É RESOLVIDO VS. LIMITES TEÓRICOS

O que nossa formalização resolve

✓ Imunidade Total a Overflow Numérico: `Int` e estoura ou depende de precisão arbitrária. Nosso autômato soma números com 1 milhão de dígitos com 100% de exatidão!

✓ Complexidade Linear Ótima O(n): uma única passada simultânea pelas strings. Cada consulta em `sd` é O(1).

✓ Determinismo Referencial: sem efeitos colaterais, variáveis globais ou mutabilidade.

✓ Prova Construtiva: o código em Haskell é a própria formalização matemática executável.

Limites & desafios teóricos

⚠ O Problema da Parada (Turing-Completude): o compilador não prova que recursões gerais terminam; aqui o término decorre da redução estrutural do tamanho das listas.

⚠ Entradas Fora do Alfabeto Numérico: `sd` cobre {'0'..'9'}. Entradas como "12a" exigiriam validação prévia ou retorno em `Maybe` para manter a totalidade.

⚠ Domínio Restrito aos Naturais (ℕ): a FSM atual modela apenas a adição de naturais. Inteiros com sinal exigiriam uma FSM de subtração e complemento.

Síntese do desafio

- Desafio Cumprido Integralmente: a soma simbólica foi executada estritamente no universo de caracteres (Char), sem qualquer tipo numérico nativo ou intermediário.
- Elegância Declarativa: o código em Haskell espelha com fidelidade a definição matemática abstrata da Máquina de Mealy.
- Poder da Abstração: a aritmética básica é apenas uma instância particular de transdutores finitos de linguagens formais.
- Agradecimentos: ao Prof. João Paulo (JP) pela provocação intelectual

Aleatório

Pergunta provável: "Por que formalizar a soma como uma Máquina de Mealy e não criar uma Mônada de Estado (State Monad) ou uma Máquina de Moore?"

A escolha pela Máquina de Mealy pura atende ao princípio da parcimônia conceitual (Navalha de Occam): fornece a menor estrutura matemática necessária, com apenas 2 estados de memória. A State Monad seria um enriquecimento categorial elegante para sequenciação (como fizemos na IC), mas a recursão estrutural com Mealy evita overhead de abstração e torna a tabela diretamente inspecionável e verificável por humanos."